

Problem Set 3 – Multi-Dimensional Packet Scrambler

Kshitiz Reddy
M.Tech AIML, School of AI, Bennett University

Source Code

```
1  """
2  Program: Multi-Dimensional Packet Scrambler
3  Purpose: Runs a data packet through a four-stage pipeline (middle-out
4           slice swap, in-place correction, memory integrity check) using
5           only built-in list operations, no external libraries.
6  Author: Kshitiz
7  """
8
9  packet = [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
10
11 # --- Stage 1: Input Validation ---
12 if packet and len(packet) >= 10:
13     print("Validation passed. Processing packet...")
14 else:
15     print("Validation failed: packet is empty or too short.")
16
17 print(f"Initial packet: {packet}")
18
19 # --- Stage 2: Middle-Out Swap ---
20 midpoint = len(packet) // 2
21 front_half = packet[:midpoint]
22 back_half = packet[midpoint:]
23 scrambled = back_half[::-1] + front_half
24
25 print("--- Stage 2: Middle-Out Swap ---")
26 print(f"Scrambled: {scrambled}")
27 print(f"Non-destructive check, id(packet) == id(front_half): "
28       f"{id(packet) == id(front_half)}")
29
30 # --- Stage 3: In-Place Correction ---
31 print("--- Stage 3: In-Place Correction ---")
32
33 middle_index = len(scrambled) // 2
34 if type(scrambled[middle_index]) is int:
35     scrambled.insert(middle_index + 1, "SYNC-BIT")
36 print(f"After Sync-Bit insertion: {scrambled}")
37
38 while 0 in scrambled:
39     scrambled.remove(0)
40 print(f"After zero removal: {scrambled}")
41
42 # --- Stage 4: Memory Integrity Check ---
43 print("--- Stage 4: Memory Integrity Check ---")
44 print(f"Original packet: {packet}")
45 print(f"Final scrambled: {scrambled}")
46
47 first, *middle, last = scrambled
48 print(f"Header: {first} Footer: {last} Body length: {len(middle)}")
49
50 # Submission Documentation:
51 # Stage 4 reprints the original packet variable, untouched since Stage 1,
```

```

52 # next to the final scrambled list. front_half, back_half, and scrambled
53 # were all built through slicing or concatenation, never through direct
54 # reference assignment, so none of them alias packet's underlying list.
55 # The id() check in Stage 2 confirms this: slicing always returns a new
56 # list object. Because every later mutation (insert, remove) runs only
57 # on scrambled, and scrambled was never the same object as packet,
58 # packet's contents at Stage 4 are guaranteed to match Stage 1 exactly.
59
60
61 def scramble(data):
62     """Stretch goal: same pipeline, returns the result instead of
63     printing it."""
64     mid = len(data) // 2
65     front, back = data[:mid], data[mid:]
66     result = back[::-1] + front
67
68     mi = len(result) // 2
69     if result and type(result[mi]) is int:
70         result.insert(mi + 1, "SYNC-BIT")
71
72     while 0 in result:
73         result.remove(0)
74
75     return result

```

Trace Log

```

1 Validation passed. Processing packet...
2 Initial packet: [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
3 --- Stage 2: Middle-Out Swap ---
4 Scrambled: [3, 0, 19, 7, 34, 5, 12, 0, 8, 21]
5 Non-destructive check, id(packet) == id(front_half): False
6 --- Stage 3: In-Place Correction ---
7 After Sync-Bit insertion: [3, 0, 19, 7, 34, 5, 'SYNC-BIT', 12, 0, 8, 21]
8 After zero removal: [3, 19, 7, 34, 5, 'SYNC-BIT', 12, 8, 21]
9 --- Stage 4: Memory Integrity Check ---
10 Original packet: [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
11 Final scrambled: [3, 19, 7, 34, 5, 'SYNC-BIT', 12, 8, 21]
12 Header: 3 Footer: 21 Body length: 7

```

Output matches the worked example in the assignment exactly: scrambled list, Sync-Bit insertion, zero removal, and the final Header/Footer/Body-length unpacking.

Submission Notes

Stage 4 reprints the original packet variable, which is never touched after Stage 1, alongside the final scrambled list. `front_half`, `back_half`, and `scrambled` are all built through slicing or concatenation, never direct reference assignment, so none of them alias the original list – the `id()` check in Stage 2 confirms slicing always returns a new object. Since every later mutation (`insert`, `remove`) only runs on `scrambled`, the original packet is guaranteed to stay exactly as it was at Stage 1.